Deliverable 2: Documentation of Our Decision

EventHub Ticketing

We chose Option A, the Minimal Context diagram, as the top-level design for EventHub Ticketing.

The main reason is fidelity to the requirements. A context diagram is meant to show agreed facts about the system's boundary, not guesses. On Call 2 the team explicitly settled several of these facts. There is one user role, because students, parents, and international fans are the same role from the system's point of view. Payment is simulated in version 1, so no payment processor sits outside the boundary. Adaeze confirmed that confirmation is on-screen only, so no email service is involved. Marcus stated that exchange rates come from a manually edited configuration file inside the system and that "there's no rate provider on the internet we call." Events are loaded from a data file at setup, and Marcus said no one needs a screen for that. Option A reflects every one of these decisions. Option B adds an external Exchange Rate Provider, which directly contradicts what the Product Manager said, and it adds an Operator role that the team never agreed to create.

The second reason is security. Kenji noted that a single user role keeps the security work simple, and he required that every piece of data crossing into the system be checked at the boundary. Option A has exactly one path into the system, so there is exactly one boundary to validate. Option B adds two more crossings: an operator interface and an inbound feed of exchange rates from a third party. Each crossing becomes something to authenticate, validate, and monitor. Without a requirement behind them, those crossings add risk and deliver no value. We also recorded Kenji's boundary validation rule directly on the EventHub system element, because Priya said the rule belongs on the context diagram itself.

The third reason is scope and cost. Adaeze set Python and Flask as a fixed constraint because the stack is cost-effective and familiar to the team. Launch traffic will mainly come from about 26,000 USF students, with peaks of a few hundred concurrent buyers during ticket releases. An external rate service would add a dependency, a potential subscription cost, and a failure mode, which is an outage at the provider, for a version 1 that does not need it. Option A keeps version 1 as small as the requirements allow.

We did not ignore what Option B gets right. It raises real questions: how exchange rates are edited, and who reads the audit log. However, both of these are still open items. Priya put the rates mechanics on the open list for the design call, and Kenji noted "we haven't said who" reads the audit log. The correct response to an open question is to record it and resolve it, not to draw an answer before the team makes the decision. We are keeping these as open questions in the requirements. If the design call assigns an operator role or chooses an external rate source, the diagram will be updated. Because the diagrams are text files under version control, that change would be reviewed the same way as any code change, which is the workflow Priya described.

Finally, the choice between a single application and separate services was deliberately left out of both diagrams. That decision concerns how EventHub is built internally, so it belongs at the C4 container level, not the context level. Priya deferred it to a written decision record, so it will be addressed in the design call and would not change this diagram either way.

In summary, Option A is the accurate, secure, and appropriately scoped picture of what the team has actually agreed. Option B describes a possible future system, but it depends on decisions that have not been made yet.